Applied Machine Learning for Business Analytics

Lecture 9: Agent: Thinking Patterns

Lecturer: Zhao Rui

Agenda

1. What are Agents
2. Thinking Patterns
    a. CoT
    b. ReACT
    c. Plan and Execute
    d. Reflection
    e. ToT
3. Practices for Building Agents

1. Agents

What is an LLM Agent

- Agent is a set of functionalities or abstractions layered on top of an LLM
  - LLM is the brain
  - Extended capabilities are added to enable LLM to interact with the external world: collect information, planning, reasoning, and taking action
- Three major components:
  - Memory
  - Tools
  - Planning

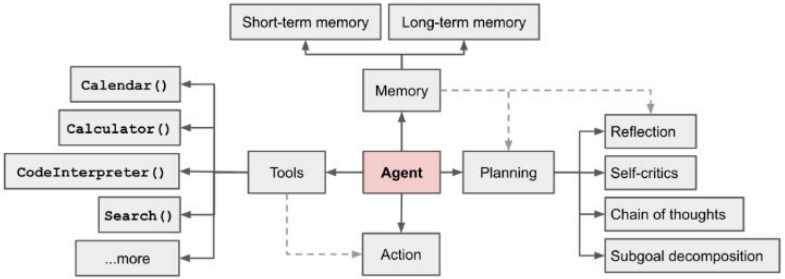


Fig. 1. Overview of a LLM-powered autonomous agent system.

Three Levels of Agent Design

Level	What It Decides	Examples
Capabilities	What the agent can do	Tool use, memory, RAG
Thinking Patterns	**How the agent reasons**	**ReAct, CoT, Reflection ←**
Orchestration Patterns	How LLM calls are structured	Prompt chaining, routing, orchestrator-worker

Three Levels, Three Questions

- Capabilities: *What can my agent act?*
  - Search engine? Database? Code execution?
  - Required for Agent
- Thinking Patterns: *How does it decide what to do?*
  - The LLM already has “reasoning” capability. But with ReAct and etc, they can think better and more controllably. (like methodology to humans)
  - Optional - Improve Reasoning
- Orchestration: *How do multiple calls coordinate?*
  - Only matters when a single LLM call is not enough.
  - Optional - Supports complex flows

⚠️ A minimal agent: LLM + one tool + a while loop

2. Thinking Patterns

Chain of Thoughts (CoT)

- The most important pattern for building real-world agents.
  - Prompt the LLM to “think step by step” before answering
  - Forces the model to use intermediate tokens as working memory
  - Dramatically improves accuracy on math, logic, multi-step reasoning

Without CoT vs. With CoT

```
[Without CoT]
Q: A store has 3 shelves × 8 items, removes 5. How many remain?
A: 19  ← model just guesses

[With CoT]
Q: Same question. Let's think step by step.
A: Step 1: 3 × 8 = 24 items total
   Step 2: 24 - 5 = 19 items remain
   Answer: 19  ← model reasons through it
```

⚠️ Limitation: Plans everything in one shot — cannot adjust if something goes wrong partway through

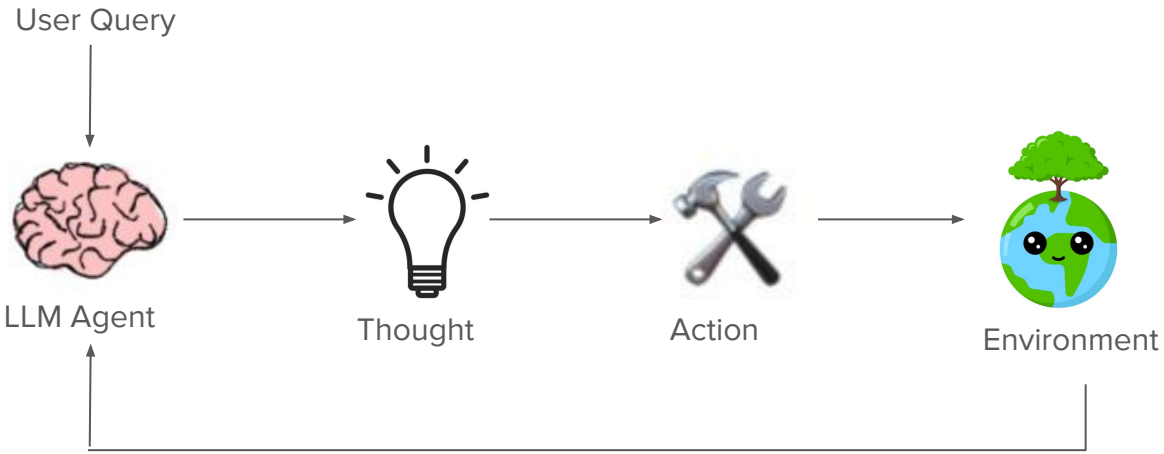
ReAct (Reason + Act)

- The most important pattern for building real-world agents
  - The agent alternates: Think-> Act->Observe->Think->....
  - Think: reason about what to do next
  - Act: use a tool or take an action
  - Observe: get the result, update understanding
  - Repeat until the task in complete

Everyday Analogy: Cooking

Cooking a new recipe:
1. Think: "Chop the onions"
2. Act: Chop them
3. Observe: "Done, looks right"
4. Think: "Heat oil in the pan"
5. Act: Heat the oil
6. Observe: "Smoking — too hot!"
7. Think: "Turn it down, add onions at lower heat"

ReAct Agent



ReAct in Action

Weather Example

```
User: "What's the weather in Singapore? Should I bring an umbrella?"

Think:  I need current weather. Let me search.
Act:    [calls weather API for Singapore]
Obs:    31°C, humidity 85%, thunderstorm warning until 6pm
Think:  Thunderstorm warning → recommend umbrella
Answer: "31°C with a thunderstorm warning until 6pm. Bring an umbrella."
```

```
while task is not complete:
    thought = LLM("What should I do next?")
    action  = LLM selects tool + params
    result  = execute_tool(action)
    add result to context
```

- Adapts in real-time to unexpected results
- Grounded in real observations, not imagination
- Model decides when to stop

⚠️ Limitation: Can wander aimlessly on complex tasks. May loop without guardrails (dead-loop).

Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models" (ICLR 2023)

Plan-and-Execute

- For complex tasks where you need a roadmap before taking action
  - Create an explicit plan first (list of steps)
  - Execute each step on at a time
  - Optionally re-plan after each step based on results

Phases

```
Phase 1 — Planning:
Task: "Research top 3 competitors, write report."
Plan:
  Step 1: Identify top 3 via web search
  Step 2: Gather pricing, features, market share
  Step 3: Create comparison table
  Step 4: Write executive summary

Phase 2 — Execution:
  [Step 1] → Found: Competitor A, B, C
  [Step 2] → Competitor B: no public pricing
             → Re-plan: check analyst reports
  [Step 3] → Table created
  [Step 4] → Summary written
```

Plan-and-Execute: Variants

- Key Properties:
  - Plan is explicit and inspectable - human in the loop
  - Separates “what to do” (strategic)
  - Execute each step on at a time
  - Optionally re-plan after each step based on results

Variant	How It Works	Best For
Plan-then-execute (rigid)	Full plan upfront, no changes	Well-understood, predictable tasks
Plan-and-replan (adaptive)	Replan after each step	Tasks with uncertainty or dependencies
Hierarchical planning	High-level plan → sub-plans per step	Very complex, multi-phase projects

⚠️ Limitation: Initial plan may be wrong. Strong commitment to a bad plan wastes efforts. Re-planning helps but adds cost and latency.

Reflection / Self-Critique

- The quality control pattern
  - After generating output, the agent reviews its own work
  - Identifies mistakes or gaps, then improves
  - Can run once or loop until quality is satisfactory
  - Key properties:
    - Catches errors the initial generation Missed
    - Effective for writing, code, analysis

3-Step Loop

```
Step 1 — Generate:
  LLM produces initial answer.

Step 2 — Reflect:
  "Review this answer. Accurate?
   Complete? Well-structured?
   List any problems."

Step 3 — Revise:
  "Here's the original + critique.
   Produce an improved version."

(Repeat Steps 2–3 until no issues found)
```

⚠️ Limitation: May miss systementiac blind spots. Works best with external ground truth (test results, human review)

Tree-of-Thought (ToT)

- For problems where multiple approaches need exploration
  - Instead of one chain of reasoning, generate multiple “thought branches”
  - Evaluate which branches are most promising
  - Continue only the best one, prune the rest
  - Key properties:
    - Explore multiple paths before committing
    - Uses LLM evaluation or voting to prune bad branches
    - Most useful for math, puzzle, and etc.
    - Effective for writing, code, analysis

Example: Make 24 from {4,6,5,10}

```
Branch A: (4+5)×... → 9×? → hard to reach 24
Branch B: (10-4)×... → 6×(6-5)=6 → no
Branch C: (6-4)×... → 2×? → hard
Branch B': 10×5-... → 50-? → dead end
Branch D: (10-6)×(5+?) → needs a 1...
Branch E: 4×5 + 10 - 6 = 24 ✓ SOLUTION
```

⚠️ Limitation: Expensive - requires many LLM calls. Overkill for straightforward tasks.

Complementary Combinations

Combination	Why It Works
ReAct + Reflection	ReAct handles execution; Reflection catches mistakes after each cycle
Plan-and-Execute + ReAct	Planning sets the roadmap; ReAct handles each step adaptively
Plan-and-Execute + Reflection	Critique and improve the plan before execution begins
CoT + Any pattern	CoT is the internal reasoning inside every single LLM call — always there

These patterns are a toolkit, not a menu. You layer them based on what the task requires.

The core skill here: know which combination matches your task's complexity.

Patterns’ Tensions

Tension	What's at Stake	Resolution
Planning vs. ReAct	Planning = structure but brittle; ReAct = flexible but may wander	Hybrid: plan loosely, execute with ReAct, replan when needed
Autonomy vs. Reliability	More patterns = more capable but less predictable	Start simple; add patterns only when task demands it
Thinking depth vs. Cost/Latency	Reflection + ToT improve quality but multiply LLM calls	Use Reflection for high-stakes output; skip for simple queries

Is This All Just Prompting

Short answer: mostly yes — but not entirely.

Component	Prompting or Code?
CoT ("think step by step")	Pure prompting
ReAct (Thought/Action/Observation)	Prompting defines format; code runs the loop + executes tools
Plan-and-Execute	Prompting generates the plan; code parses + dispatches steps
Reflection	Prompting asks for critique; code manages generate → reflect → revise loop
Tool execution	LLM decides to call (prompting); code executes the API call
Memory & context management	Code manages what enters the context window

Prompts define the behavior. Code defines the control flow.

The General Architecture

```
while task is not complete:                  ← Code (the loop)
    context = assemble_relevant_info()       ← Code (context engineering)
    response = LLM(context)                  ← Prompting (thinking pattern+context)
    tool_calls = parse(response)             ← Code (parsing)
    results = execute_tools(tool_calls)      ← Code (tool execution)
    update_memory_and_state(results)         ← Code (state management)
```

- As models get smarter, prompting becomes less prescriptive
- But engineering around context, tools, and state remains essential
- The field is shifting: "prompt engineering" → "context engineering"
- Bottleneck = what information reaches the model, not how you phrase instructions

Context engineering: the new frontier.

Taxonomy is Leaky (but it is ok)

Planning is the best example — it can be implemented three different ways:

Implementation	Category
"Think step by step" inside one LLM call	Thinking pattern (reasoning)
LLM outputs a plan → code dispatches each step	Orchestration pattern (control flow)
LLM calls an external planner tool (e.g., PDDL)	Capability (tool use)

- Similarly: "tool use" is listed as a pattern by some authors but is really a capability
- The field is young — terminology hasn't settled
- Don't get stuck on taxonomy

Focus on the practical question: "For this task, what combination of thinking, tools, and control flow gives me the best result?"

When to use what

Is the task simple and single-step?

→ YES: Single LLM call with good context. No patterns needed.
→ NO: Continue...

Does it need real-world information or actions?

→ YES: Use ReAct (thinking + acting in a loop)
→ NO: Use CoT or Plan-and-Execute

Is it complex with many interdependent steps?

→ YES: Use Plan-and-Execute (+ optional re-planning)
→ NO: ReAct is sufficient

Is output quality critical (reports, code, analysis)?

→ YES: Add Reflection as a quality check
→ NO: Skip it to save cost/time

Multiple valid approaches, unsure which is best?

→ YES: Consider Tree-of-Thought
→ NO: A single reasoning chain is fine

Golden rule: Start with the simplest approach that works. Add complexity only when you have evidence it improves results.

Key Takeaways

- Thinking patterns are how agents reason - not magic, just structured prompting + code
- Each pattern addresses the failure of the simpler one before it
  - **CoT**: show ur work. **ReAct**: interact with the world. **Plan and Execute**: make a roadmap. **Reflection**: check your own work. **ToT**: ensemble all works.
- In practice, you combine them. The skill is matching the combination of the task
- Start simple. Add patterns only when you have evidence they improve results.

3. Practices behind Building Agents

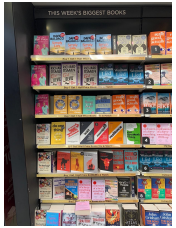
Three Levels of Agent Design

Level	Agent World	Human World	Requires
Capabilities	Tools, memory, RAG	Degrees, Skills, Capital,..	**Always**
Thinking Patterns	ReAct, CoT, Reflection	Self-help books, mental frameworks	When needed
Orchestration Patterns	Multi-agent, Chaining, routing	Team management, org structure	Last Resort

Only **capabilities** is strictly required. Without tools, memory, or retrieval, an LLM is just an language model. Thinking patterns and orchestration are **optional enhancements** - add them only when simpler approaches fall short.

Human Career Analogy

- Capabilities
  - Those are our hard resources
  - Without tools, the agent can not act
- Thinking Patterns
  - Like mental frameworks, do not add resources but teach you to use resources better
  - Someone can succeed without reading a single self-help book, like LLM can work with implicit reasoning patterns
- Orchestration
  - It only matters at scale. We need to build a team by delegating to different roles and designing workflows
  - Like a solo freelancer does not need an org chart. A simple agent also do not need multi-agent coordination



When to Add Each Level

- LLM + Tools Only
  - Just capabilities. An augmented LLM with retrieval, tools, and memory. No fancy patterns - just a well-prompted model with access to the right resources

Handle most real-world tasks

- + Thinking Patterns
  - Add React or advanced thinking patterns when tasks require multi-step reasoning, adapting to intermediate results, of self-checking output quality

When reasoning quality matters

- + Orchestration
  - Add chaining, routing, or multi-agent patterns only when a single agent loop genuinely can't handle the task's complexity.

Only for genuinely complex workflows

Start with simple prompts, optimize them with comprehensive evaluation, and add multi-step agentic systems only when simpler solutions fall short.

Antropic, "Building Effective Agents"

Antropic Five Orchestration Patterns

When you need Level 3, those are composable building blocks - from simple -> complex:

Prompt Chaining

Sequential steps, each LLM can builds on the previous. Add “gates” to validate between steps.

Routing

Classify input, dispatch to a specialized handler. Like a switch statement driven by an LLM

Parallelization

Run subtasks concurrently or run the same task multiple times (voting)

Orchestrator-Worker

A central LLM dynamically decides subtasks and delegates to workers. Flexible but comlex.

Evaluator-Optimizer

One LLM generates, another evaluates. Loop until quality threshold is met. Iterative refinement.

Wrap it up

- Agent Design Equation
  - Capabilities X Thinking X Orchestration
- Three Guiding Principles
  - Keep it simple
    - Start small, build modularly. Add complexity only when it measurably improves outcomes
  - Make reasoning visible
    - Let users and developers see how the agent plans and decides. Transparency builds trust
  - Design tools carefully
    - Well-scoped, well-documents tools matter more than clever orchestration patterns

Next Class: LLM: Memory

Appendix

Build ReAcT with LangChain

```
from langchain import hub
from langchain.agents import AgentExecutor, create_react_agent
from langchain_community.tools.tavily_search import TavilySearchResults
tools = [TavilySearchResults(max_results=1)]
# Get the prompt to use - you can modify this!
prompt = hub.pull("hwchase17/react")
# Construct the ReAct agent
agent = create_react_agent(llm, tools, prompt)
# Create an agent executor by passing in the agent and tools
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

Prompt Template

```
prompt = hub.pull("hwchase17/react")
```

input_variables=['agent_scratchpad', 'input', 'tool_names', 'tools']
input_types={} partial_variables={}
metadata={'lc_hub_owner': 'hwchase17', 'lc_hub_repo': 'react', 'lc_hub_commit_hash': 'd15fe3c426f1c4b3f37c9198853e4a86e20c425ca7f4752ec0c9b0e97ca7ea4d'}
template='**Answer the following questions as best you can. You have access to the following tools:\n\n{tools}\n\nUse the following format:\n\nQuestion: the input question you must answer\nThought: you should always think about what to do\nAction: the action to take, should be one of [{tool_names}]\nAction Input: the input to the action\nObservation: the result of the action\n... (this Thought/Action/Action Input/Observation can repeat N times)\nThought: I now know the final answer\nFinal Answer: the final answer to the original input question\n\nBegin!\n\nQuestion: {input}\nThought:{agent_scratchpad}**'

```
Answer the following questions as best you can. You have access to the following tools:

{tools}

Use the following format:

Question: the input question you must answer
Thought: you should always think about what to do
Action: the action to take, should be one of [{tool_names}]
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Action Input/Observation can repeat N times)
Thought: I now know the final answer
Final Answer: the final answer to the original input question

Begin!

Question: {input}
Thought:{agent_scratchpad}
```

ReAct Agent-Trace

```
agent_executor.invoke({"input": "What is the headquarters of the company where the lecturer of BT5153 works? BT5153 is a module offered by NUS, MBSA.?"})
✓ 17.3s                                                                                                                                Python

> Entering new AgentExecutor chain...
To find the headquarters of the company where the lecturer of BT5153 works, I first need to identify the lecturer of BT5153 at NUS, MBSA. Then, I can find out the company they are associated with and determine its headquarters.

Action: tavily_search_results_json
Action Input: "BT5153 NUS MBSA lecturer"
[{'url': 'https://www.coursehero.com/sitemap/schools/2652-National-University-of-Singapore/courses/18558930-BT5153/', 'content': 'Lecturer: ZHAO Rui Logistics Check course website freq. ... A0049228B_#1.pdf. BT5153: Applied Machine L

Action: tavily_search_results_json
Action Input: "Zhao Rui company affiliation"
[{'url': 'https://ieeexplore.ieee.org/author/37085537953', 'content': "Affiliations: [Company of Pluang, Singapore]. Author Bio: Authors' photographs and biographies not available at the time of publication. Rui Zhao. Affiliation. Cc

Action: tavily_search_results_json
Action Input: "Pluang headquarters location"
[{'url': 'https://www.cbinsights.com/company/pluang', 'content': "Pluang is an all-in-one investment platform that operates in the financial services sector, offering a range of asset classes for investment. Use the CB Insights Platf

Final Answer: The headquarters of the company where the lecturer of BT5153 works, Pluang, is in Jakarta, Indonesia.

> Finished chain.

{'input': 'What is the headquarters of the company where the lecturer of BT5153 works? BT5153 is a module offered by NUS, MBSA.?',
 'output': 'The headquarters of the company where the lecturer of BT5153 works, Pluang, is in Jakarta, Indonesia.'}
```

ReAct Agent-Trace

> Entering new AgentExecutor chain...

To find the headquarters of the company where the lecturer of BT5153 works, I first need to identify the lecturer of BT5153 at NUS, MBSA. Then, I can find out the company they are associated with and determine its headquarters.

Action: tavily_search_results_json
Action Input: "BT5153 NUS MBSA lecturer"
[{'url': 'https://www.coursehero.com/sitemap/schools/2652-National-University-of-Singapore/courses/18558930-BT5153/', 'content': 'Lecturer: ZHAO Rui Logistics Check course website freq. ... A0049228B_#1.pdf. BT5153: Applied Machine Learning for Business Analytics (Spring 2021) Text Classification Kaggle Project Report HUANG Sixuan (A0049228B) Introduction 2. Base Case Description This project is a text classification problem for the comments posted on two Q&A.'}]I found that the lecturer for BT5153 at NUS, MBSA is Zhao Rui. Now, I need to identify the company where Zhao Rui works and determine its headquarters.

Action: tavily_search_results_json
Action Input: "Zhao Rui company affiliation"
[{'url': 'https://ieeexplore.ieee.org/author/37085537953', 'content': "Affiliations: [Company of Pluang, Singapore]. Author Bio: Authors' photographs and biographies not available at the time of publication. Rui Zhao. Affiliation. Company of Pluang, Singapore. Publication Topics Convolutional Neural Network,Deep Learning,Fault Diagnosis,Time Step,Attention Mechanism,Complex Models,Convolutional Layers"}]Thought: The search results indicate that Zhao Rui is affiliated with the company Pluang in Singapore. Now, I need to determine the headquarters of Pluang.

Action: tavily_search_results_json
Action Input: "Pluang headquarters location"
[{'url': 'https://www.cbinsights.com/company/pluang', 'content': "Pluang is an all-in-one investment platform that operates in the financial services sector, offering a range of asset classes for investment. Use the CB Insights Platform to explore Pluang's full profile. ... Headquarters Location. Jalan M.H. Thamrin No. 15 The Plaza Office Tower. Jakarta, 10350, Indonesia +62-0218063 0065. Suggest an edit"}]The headquarters of Pluang is located at Jalan M.H. Thamrin No. 15, The Plaza Office Tower, Jakarta, Indonesia.

Final Answer: The headquarters of the company where the lecturer of BT5153 works, Pluang, is in Jakarta, Indonesia.

> Finished chain.

Agent Failure

- The more complex a task an agent performs, the more possible failure points there are
- Agents can make mistakes from the following aspects:
  - Planning
  - Tool execution
  - Efficiency

```python
# Create an agent executor by passing in the agent and tools
agent_executor = AgentExecutor(agent=agent, tools=tools)
# Run the agent
agent_executor.invoke({"input": "What is the headquarters of the company where the lecturer of BT5153 works? BT5153 is a module offered by NUS, MBSA.?"})
```

✓ 16.2s Python

```
{'input': 'What is the headquarters of the company where the lecturer of BT5153 works? BT5153 is a module offered by NUS, MBSA.?',
 'output': 'Rui Zhao is affiliated with State Grid Liaoning Electric Power Supply Co., Ltd, headquartered in Anshan, China.'}
```